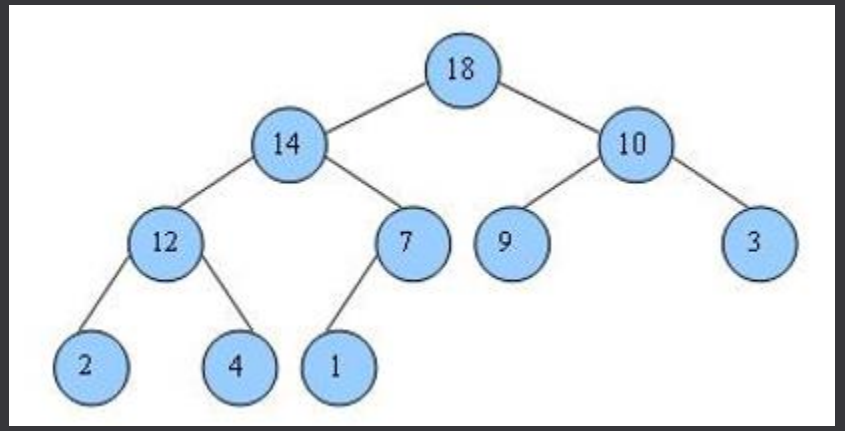
Algoritmusok és adatszerkeztek I.

9. gyakorlat

Tartalom



- Bináris keresőfa algoritmusok
- Kupac adatszerkezet
- Kupac ábrázolása
- Kupac műveletek
- Prioritásos sor megvalósítása kupaccal
- Kupac rendezés szemléltetése

Bináris keresőfa algoritmusok

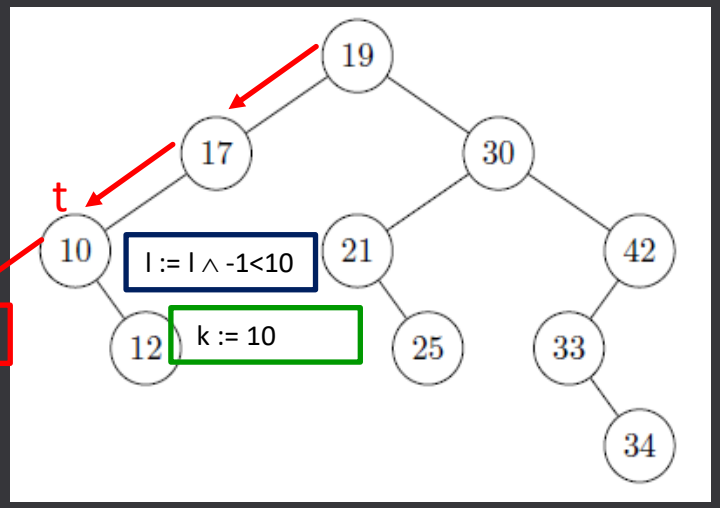
1. Készítsünk rekurzív algoritmust, mely egy láncoltan ábrázolt bináris fáról eldönti, hogy bináris keresőfa-e. A fa Node típusú elemekből áll, a kulcsok természetes számok.
2. Adott egy A[1..n] tömb, elemei tetszőleges típusú kulcsok, a tömb szigorúan monoton növekvő. Készítsünk rekurzív algoritmust, mely a tömb alapján felépíti az optimális alakú bináris keresőfát. A fát három pointerrel (Node3 típusú elemekkel) kell ábrázolni. A szükséges elemeket menet közben hozza létre az algoritmus, eredményül a fa gyökerére mutató pointert adjon.

- Készítsünk rekurzív algoritmust, mely egy láncoltan ábrázolt bináris fáról eldönti, hogy bináris keresőfa-e. A fa Node típusú elemekből áll, a kulcsok természetes számok.
- Első megoldás

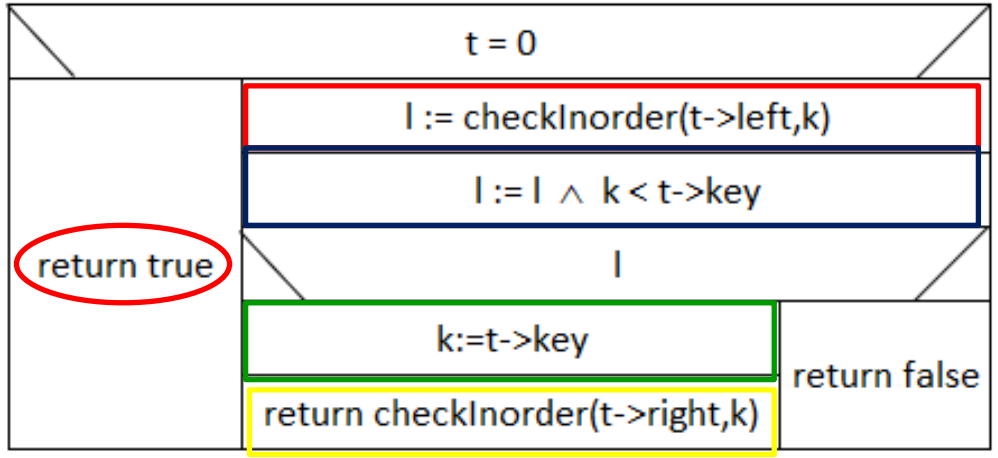
checkBinSearchTree(t: Node*):B

k:=-1
return checkInorder(t,k)

t k=-1



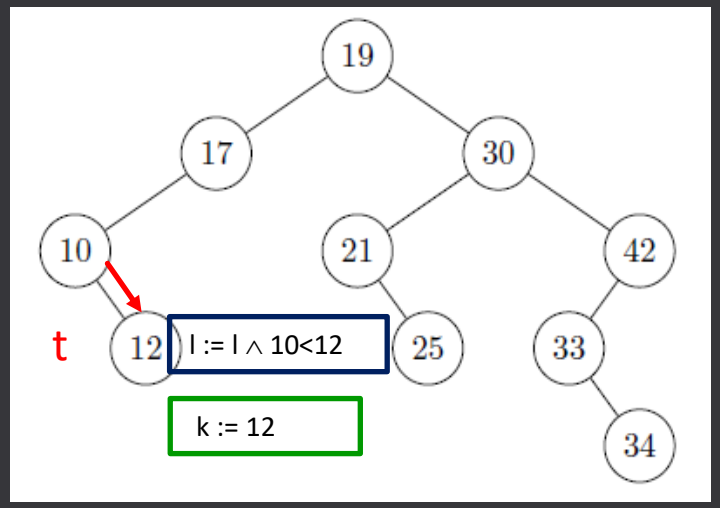
checkInorder(t:Node*, &k:Z) : B



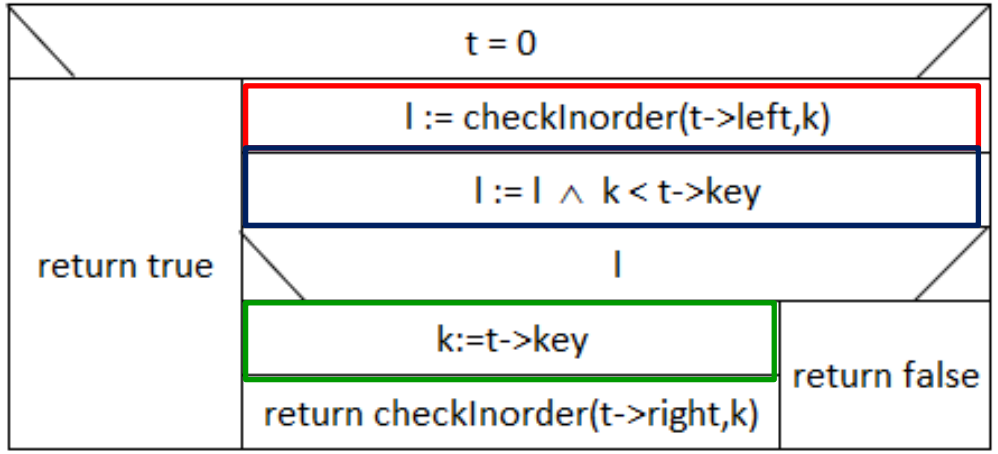
- Készítsünk rekurzív algoritmust, mely egy láncoltan ábrázolt bináris fáról eldönti, hogy bináris keresőfa-e. A fa Node típusú elemekből áll, a kulcsok természetes számok.
- Első megoldás

checkBinSearchTree(t: Node*):B

k:=-1
return checkInorder(t,k)



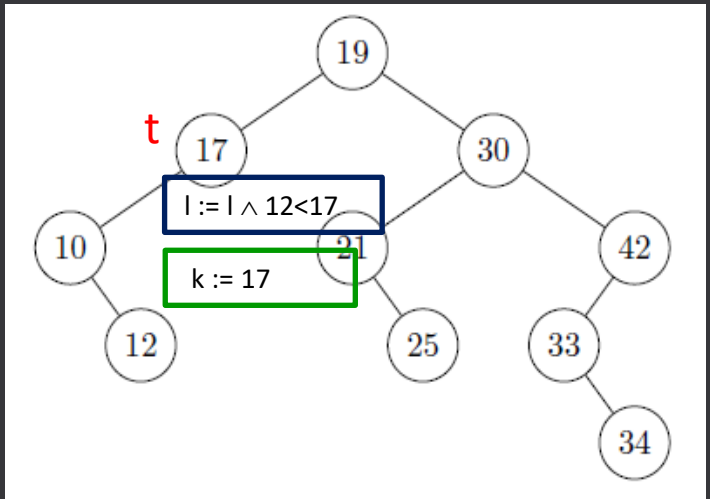
checkInorder(t:Node*, &k:Z) : B



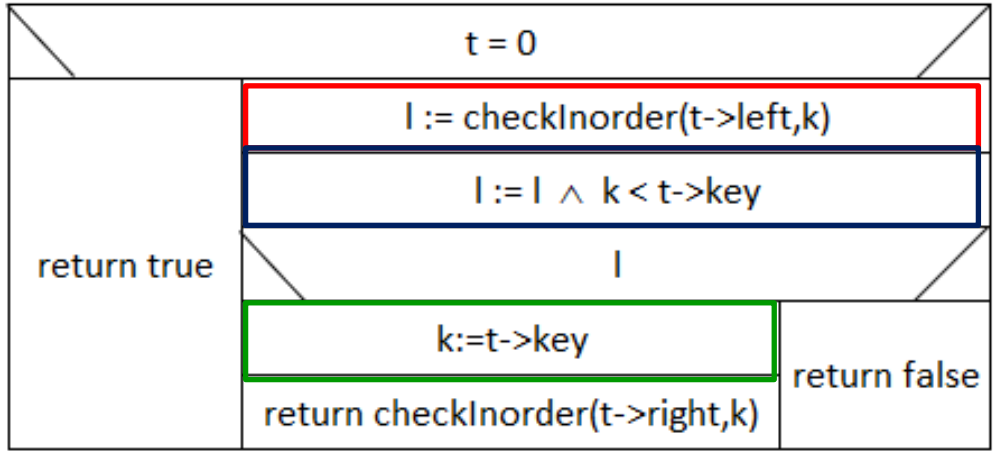
- Készítsünk rekurzív algoritmust, mely egy láncoltan ábrázolt bináris fáról eldönti, hogy bináris keresőfa-e. A fa Node típusú elemekből áll, a kulcsok természetes számok.
- Első megoldás

checkBinSearchTree(t: Node*):B

k:=-1
return checkInorder(t,k)



checkInorder(t:Node*, &k:Z) : B

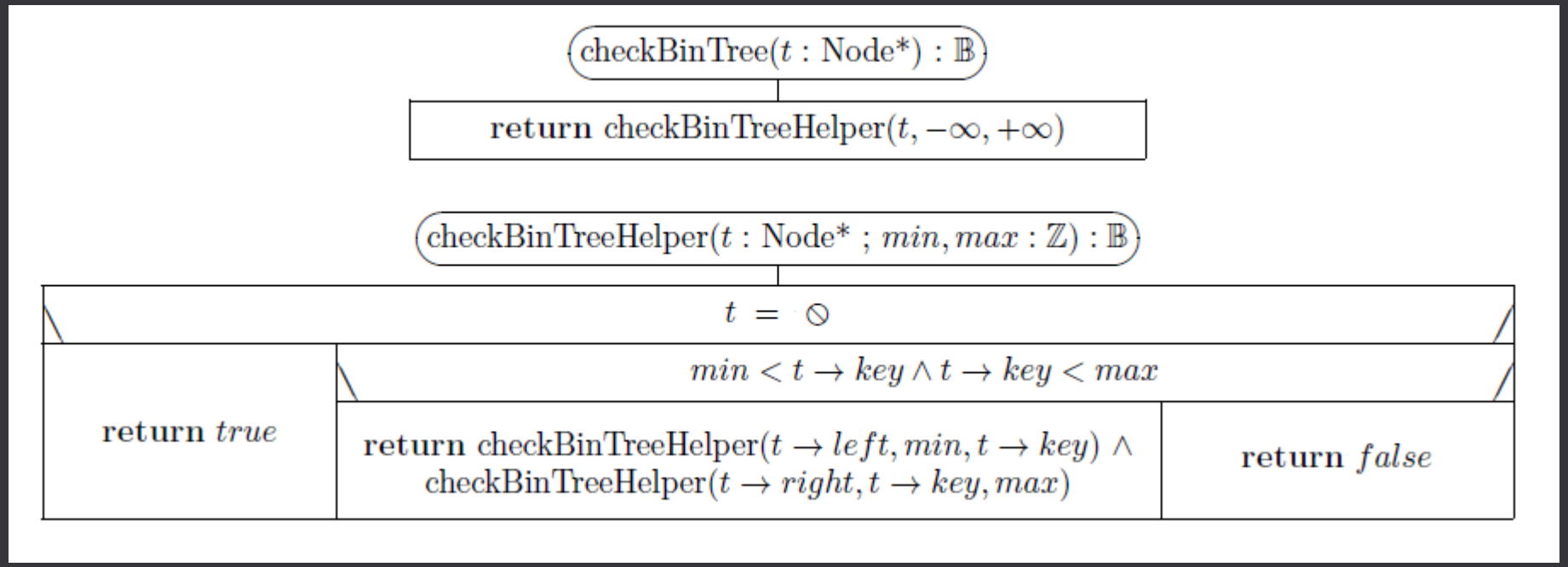


és így tovább…

Műveletigény: O(n)
n a fa csúcsaiak száma

- Készítsünk rekurzív algoritmust, mely egy láncoltan ábrázolt bináris fáról eldönti, hogy bináris keresőfa-e. A fa Node típusú elemekből áll, a kulcsok természetes számok.
- Egy másik érdekes megoldás, feltesszük most, hogy a kulcsok egész számok, és rendelkezünk két extremális értékkel:

  -∞ (kisebb az összes kulcsnál) +∞ (nagyobb az összes kulcsnál):



Műveletigény: O(n) ,ahol n a fa csúcsaiak száma

- Adott egy A[1..n] tömb, elemei tetszőleges típusú kulcsok, a tömb szigorúan monoton növekvő. Készítsünk rekurzív algoritmust, mely a tömb alapján felépíti az optimális alakú bináris keresőfát. A fát három pointerrel (Node3 típusú elemekkel) kell ábrázolni. A szükséges elemeket menet közben hozza létre az algoritmus, eredményül a fa gyökerére mutató pointert adjon.
- Háttér: a bináris keresőfa alakja nagyon el tud romlani, egyes ágai túl hosszúra nőhetnek (akár listává torzulhat a fa), így elveszíti a benne történő keresés a hatékonyságát. Erre a problémára tudnak megoldást adni az „önkiegyensúlyozó” bináris keresőfák, például az AVL fák vagy a piros-fekete fák. A következő félévben fogjuk tanulni az AVL fákat. Érdekes viszont a következő ötlet: ha nagyon elromlik a fa alakja, járjuk be inorder bejárással, írjuk egy tömbbe a rekordokat, majd az így kapott szigorúan monoton növekvően rendezett tömbből építsük fel újra bináris keresőfát úgy, hogy alakja optimális legyen.
- Megoldási ötlet: építsük fel rekurzívan a következő módon: gyökérnek vegyük a tömb középső elemét, majd a tőle balra és jobbra lévő elemekből hasonló módon építsünk bináris keresőfát, és csatoljuk be a fákat a gyökér alá. Ezt folytatjuk rekurzívan, míg a levelekig nem érünk. Láttuk, hogy bináris keresőfák esetén szükség lehet a szülő pointerre is, így a fát Node3 elemekből építjük, úgy, hogy a szülő pointert is beállítjuk.

Lássuk a megoldást:

BinkerfaÉpít(A/1: T[n], e:N, u:N) : Node3*

k:=⌊ (e+u)/2 ⌋	
p:= new Node3(A[k],0)	
e < k	
p->left:=BinkerfaÉpít(A,e,k-1)	// p->left:=0 skip
p->left->parent:= p	
k < u	
p->right:=BinkerfaÉpít(A,k+1,u)	// p->right:=0 skip
p->right->parent:= p	
return p	

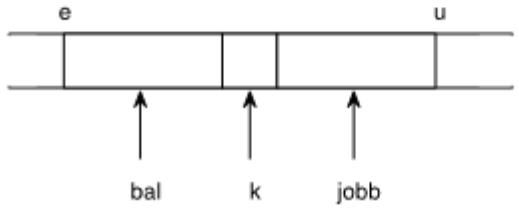
Node3 konstruktora olyan elemet hoz létre, melynek kulcsa: A[k],
left és *right* pointerei nullák, a *parent* pointert megadhatjuk, itt most 0-val hívjuk:

p ->

0		
0	A[k]	0

Ha e=k, vagy k=u, akkor a most létrehozott csúcsnak nem lesz bal-, illetve jobb részfája, a pointert nullára állíthatjuk, de felesleges, mert a konstruktor ezt már megtette.

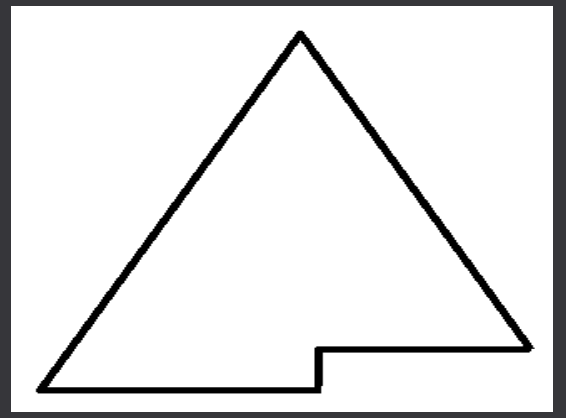
A felezés ábrája:

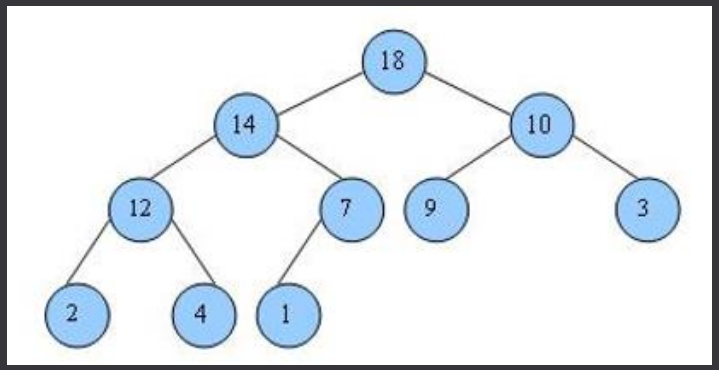


Hívás: Node3* t:=BinkerfaÉpít(A,1,n)
Műveletigény $\Theta(n)$

Kupac fogalma

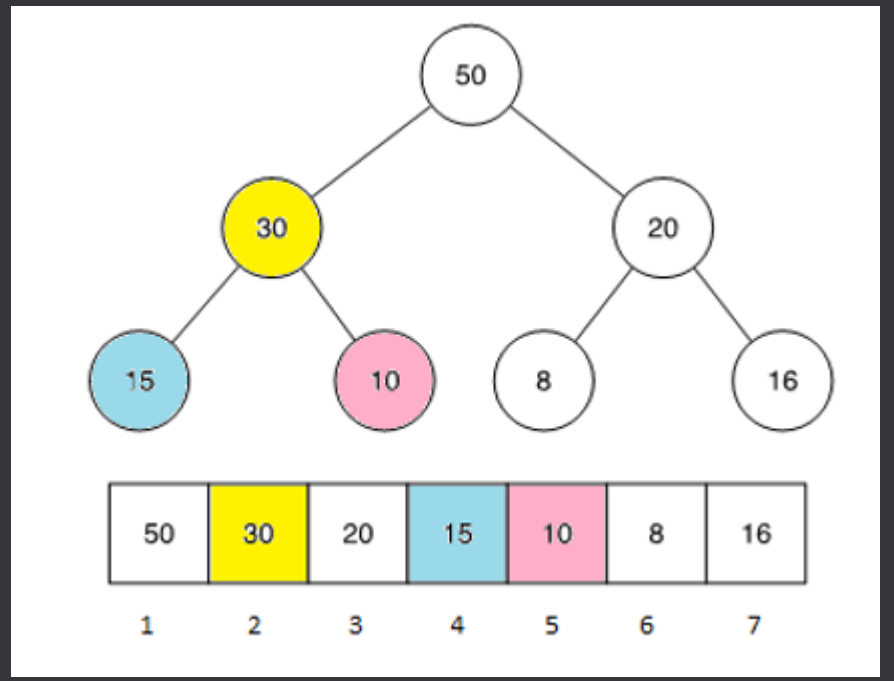
- szigorúan bináris fa: a fa minden belső pontjának két gyereke van.
- teljes bináris fa: olyan szigorúan bináris fa, ahol minden levél azonos szinten helyezkedik el.
- majdnem teljes bináris fa: olyan teljes bináris fa, melynek legalsó (levél) szintjéről elhagytunk néhány levelet (de nem az összeset).
- majdnem teljes balra tömörített bináris fa: majdnem teljes bináris fa, de levelek csak a legalsó szintről, a jobb oldalról hiányozhatnak. Ezeket szintfolytonos fáknak is nevezzük.
- kupac: *maximum vagy minimum kupac lehet.*
  - Maximum kupac: egy *majdnem teljes, balra tömörített* bináris fa, melynek minden belső pontjára teljesül, hogy a *belső pont kulcsa nagyobb vagy egyenlő a gyerekei kulcsánál.*
  - Így a maximum kupac tetején (a fa gyökerében) mindig az egyik legnagyobb elem található.
  - Minimum kupac hasonlóan: a szülő kulcs kisebb vagy egyenlő a gyerekei kulcsánál.





Kupac ábrázolása

- A szintfolytonos bináris fákat, így a kupacokat is tömbbel ábrázoljuk. (Szokás ezt az ábrázolást *„bináris fák aritmetikai ábrázolásának”* is nevezni.) A szintfolytonos elhelyezés következménye, hogy a fában való navigálás a tömb indexeinek segítségével történik.
- Ha a tömböt 1-től indexeljük, akkor:
- Legyen a csúcs indexe: *i*
  - Bal gyerekének indexe: *2*i*
  - Jobb gyerekének indexe: *2*i+1*
  - Szülő indexe: $\left\lfloor \frac{i}{2} \right\rfloor$ (i/2 alsó egész része)

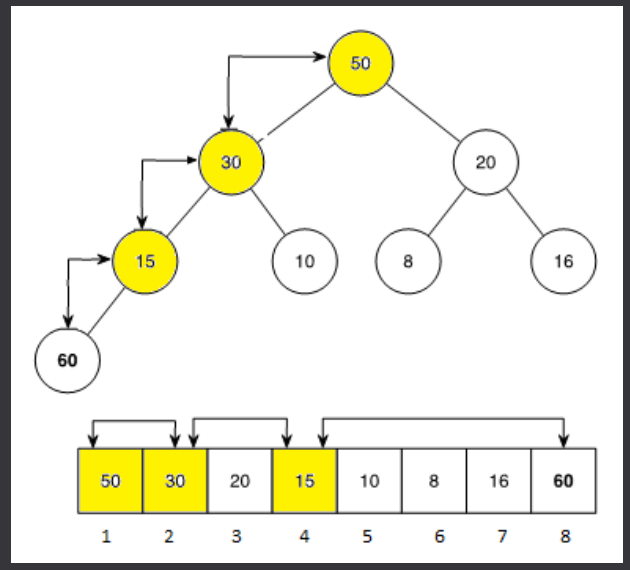


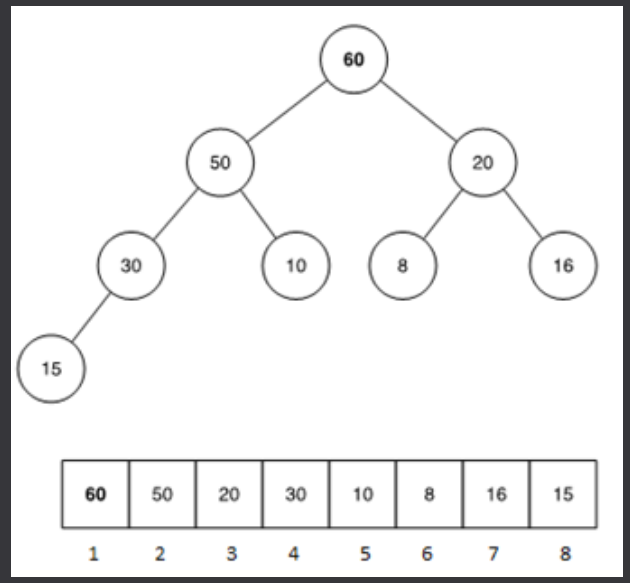
Kupac műveletei

- Két nevezetes művelettel foglalkozunk:
  - Beszúrás
  - Maximális elem kivétele
- A műveleteknek fenn kell tartania a kupac tulajdonságait: az alakjára, és a benne tárolt kulcsokra vonatkozó feltételeket.
- A kupacot tipikusan egy tömbben ábrázoljuk: A/1:T[n]
- Általában nem teljesen foglalja el az őt ábrázoló tömböt: 1..m ($m \leq n$) indexeken tároljuk a kupac elemeit, m+1..n indexeken „üres” helyek vannak, melyet elfoglalhat a kupacba beszúrandó elem.
- Lehetnek azonos kulcsok a kupacban!

Beszúrás

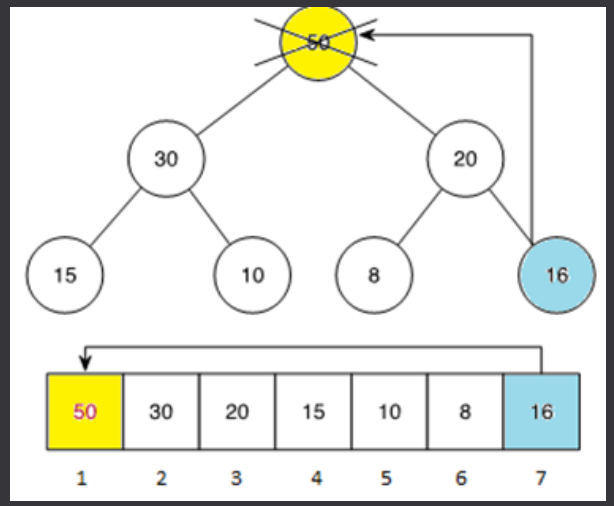
- A 60-as elem beszúrása: mivel a levelek szintje tele van, egy új szint keletkezik, és annak legbaloldalibb eleme lesz a 60.
- Majd az új elem addig **emelkedik,** míg a kupac tulajdonság helyre nem áll, azaz helyet cserél a szülőjével mindaddig, míg a beszúrt elem kulcsa nagyobb, mint a szülőjének kulcsa, vagy fel nem ér a kupac tetejére.
- Ez legfeljebb annyi cserét jelent, mint a kupac magassága, azaz $\lceil \log_2 n \rceil$.

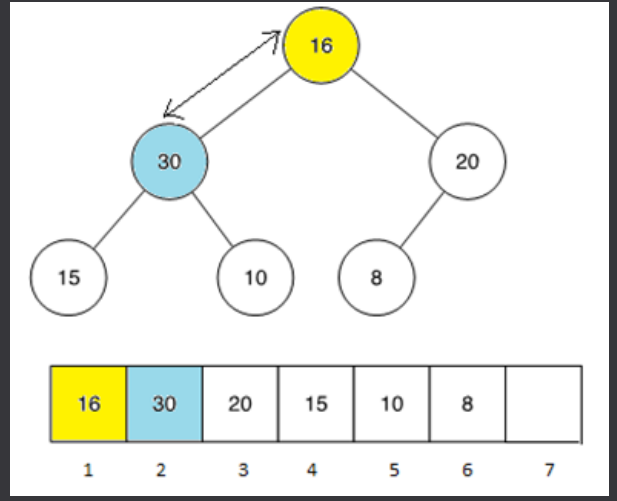


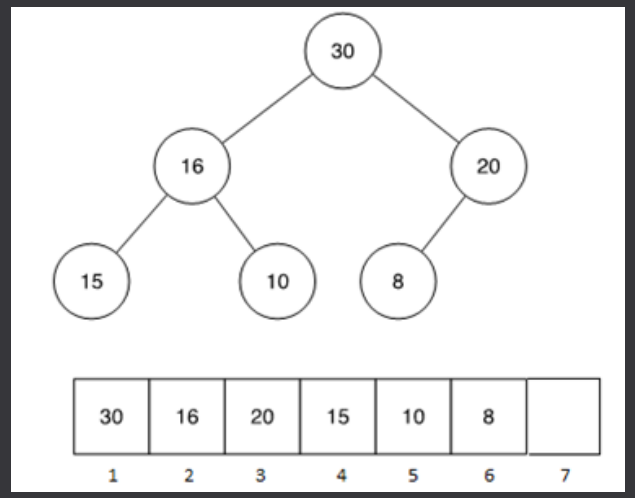


Maximális elem törlése

- a maximális kulcs eltávolításakor, helyére a fa legalsó szintjének legjobboldalibb levele kerül, azaz a tömbben a kupachoz tartozó legutolsó elem (hogy a fa megtartsa balra-tömörítettégét).
- Ezután következik az úgynevezett süllyesztés: a kulcs addig süllyed lefelé a kupacban, míg kisebb, mint a nagyobbik gyereke (ha két gyereke van). Ezért az algoritmus kiválasztja a nagyobbik gyereket, és ha a süllyesztendő kulcs kisebb nála, akkor helyet cserélnek.
- A süllyesztés addig tart, míg a süllyesztendő nagyobb vagy egyenlő lesz, mint a nagyobbik gyereke, vagy leérünk a kupac aljára.
- Lépésszám legfeljebb $\lceil \log_2 n \rceil$.







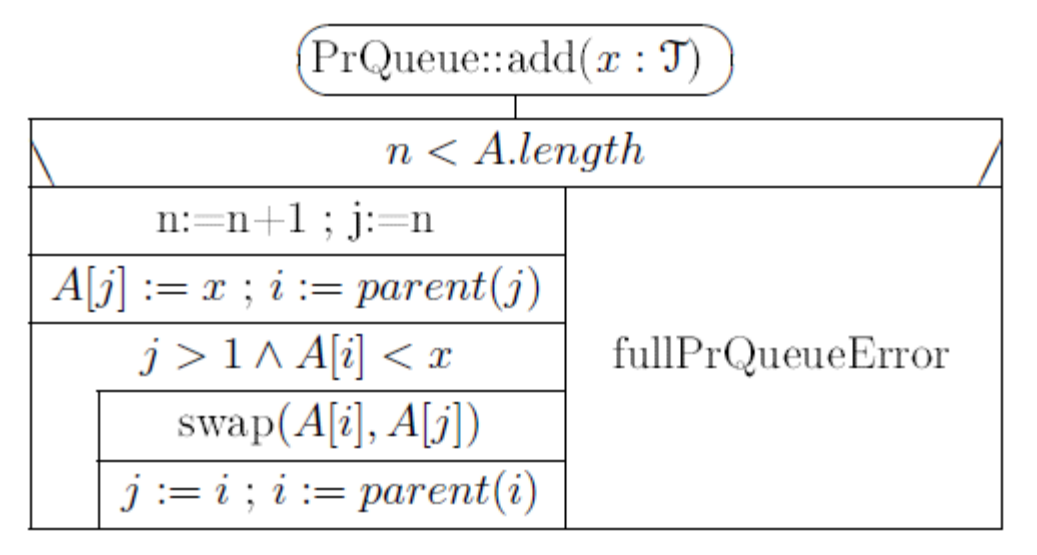
Prioritásos sor ábrázolása kupaccal

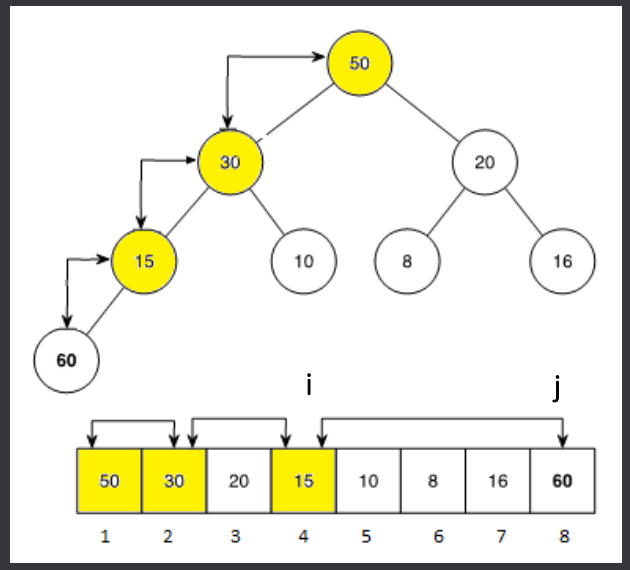
- Beszélhetünk maximum-, vagy minimum prioritásos sorról. Maximum prioritásos sor esetén mindig a legnagyobb prioritású elemet tudjuk kivenni, a minimum prioritásos sor esetén pedig a legkisebbet. Mindkettő fajta ábrázolható kupaccal.
- Itt most a maximum prioritásos sort fogjuk tanulmányozni, amit egy maximum kupaccal ábrázolunk.

PrQueue
- $A/1 : \mathcal{T}[]$ // $\mathcal{T}$ is some known type - $n : \mathbb{N}$ // $n \in [0..A.length]$ is the actual length of the priority queue

+ PrQueue($m : \mathbb{N}$){ $A :=$ **new** $\mathcal{T}[m]$; $n := 0$ } // create an empty priority queue
+ add($x : \mathcal{T}$) // insert x into the priority queue
+ remMax():$\mathcal{T}$ // remove and return the maximal element of the priority queue
+ max():$\mathcal{T}$ // return the maximal element of the priority queue
+ isFull() : $\mathbb{B}$ {**return** $n = A.length$}
+ isEmpty() : $\mathbb{B}$ {**return** $n = 0$}
+ $\sim$ PrQueue() { **delete** A }
+ setEmpty() {$n := 0$} // reinitialize the priority queue

add(x:T) művelet





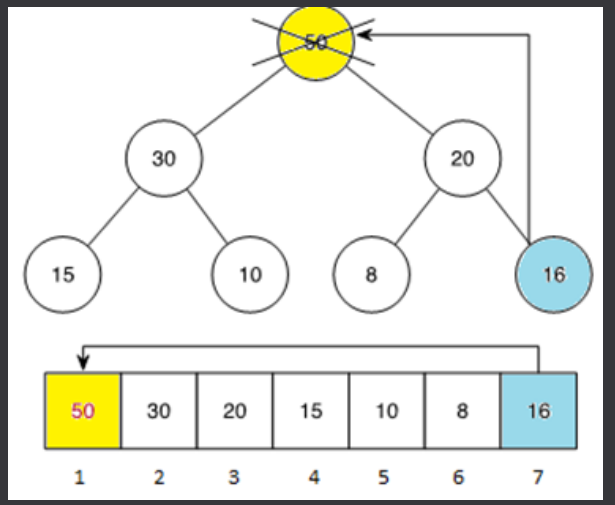
n=7

n=8

remMax():T művelet

PrQueue::remMax() : $\mathcal{T}$

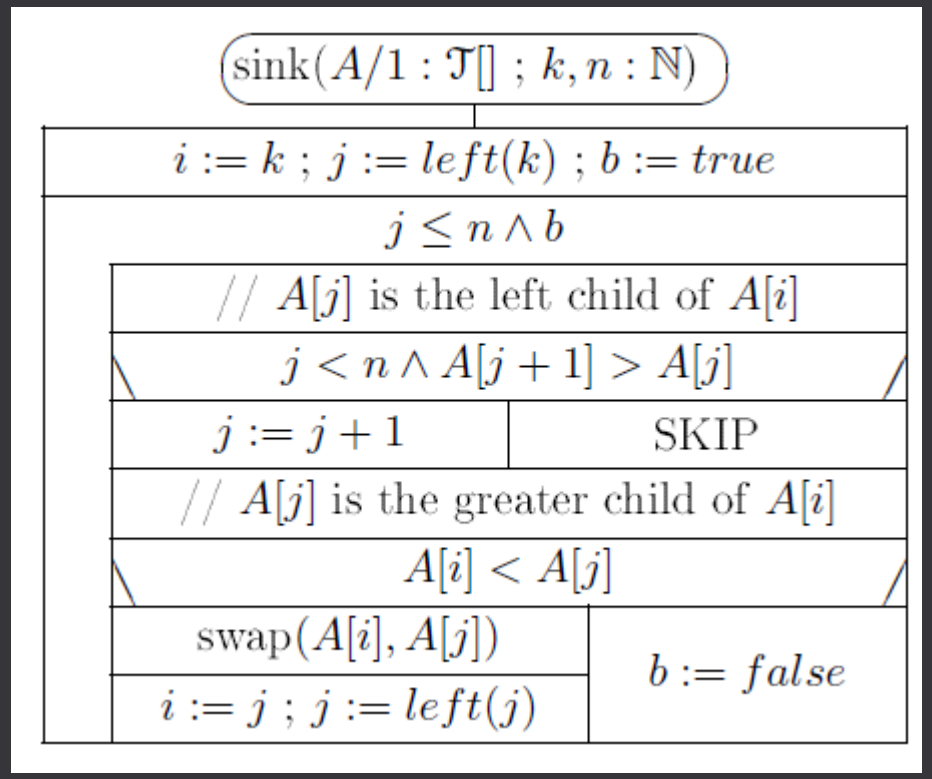
$n > 0$	
$max := A[1]$	emptyPrQueueError
$A[1] := A[n]$	
$n := n - 1$	
$\text{sink}(A, 1, n)$	
return max	

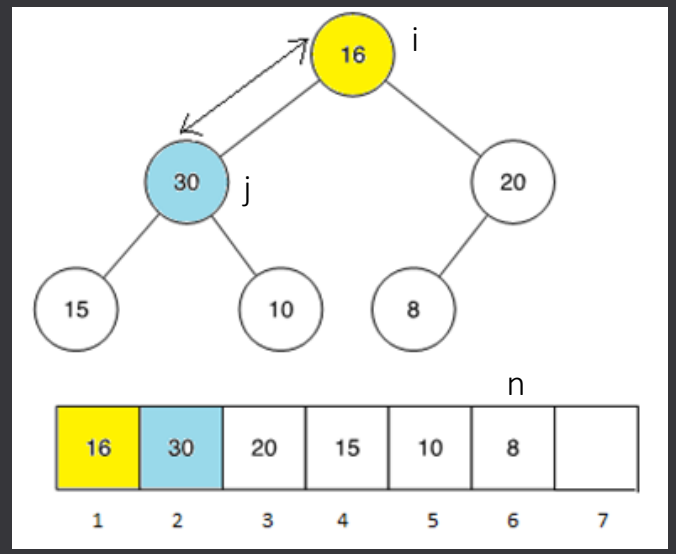


n=7

n=6

sink(A/1:T[];k,n:N) művelet

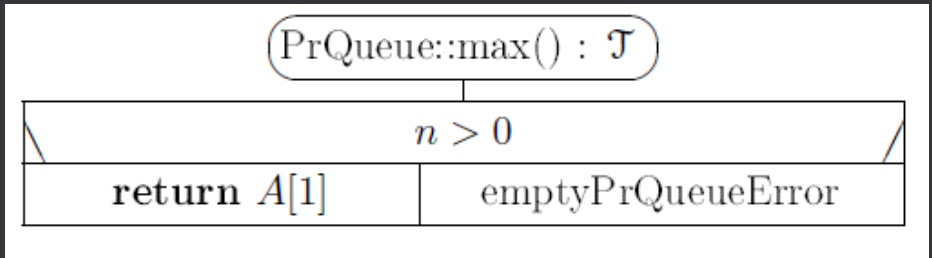




Sink(A,1,6)

Prioritásos sor különféle megvalósításainak összehasonlítása

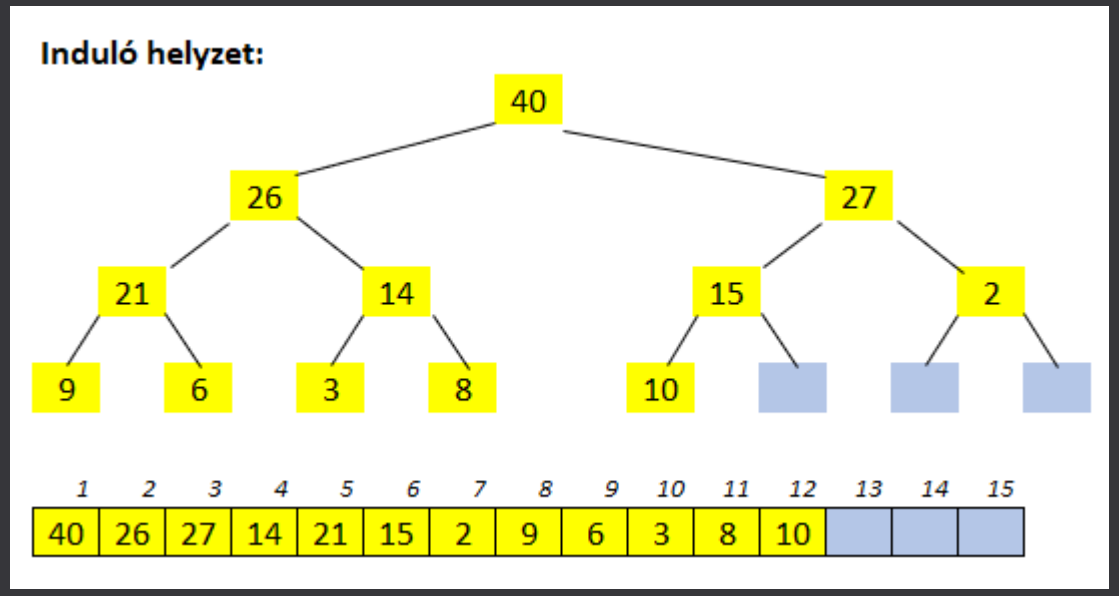
Maximum prioritásos sor megvalósításainak összehasonlítása			
	add(x)	**remMax()**	**max()**
rendezetlen tömb *ha a maximális elem indexét nyilvántartjuk*	$\Theta(1)$	$\Theta(n)$	$\Theta(1)$
rendezett tömb *növekvően rendezett*	$O(n)$	$\Theta(1)$	$\Theta(1)$
maximum kupac	$O(\log n)$	$O(\lg n)$	$\Theta(1)$



max():T művelet

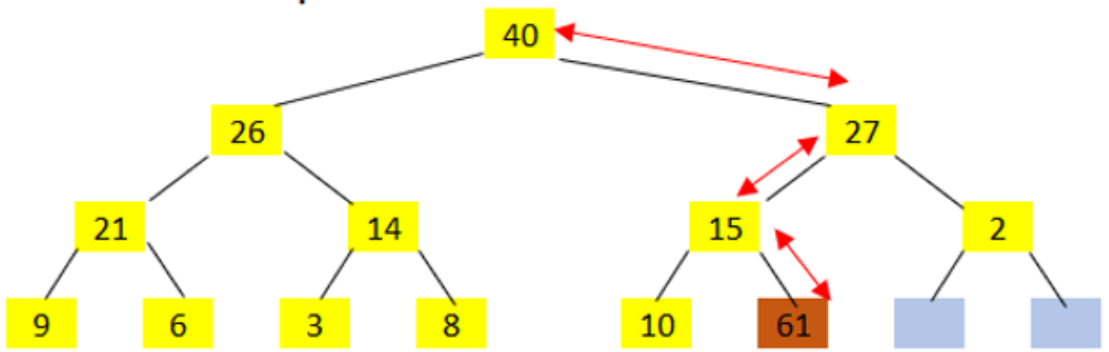
Egy feladat megoldása

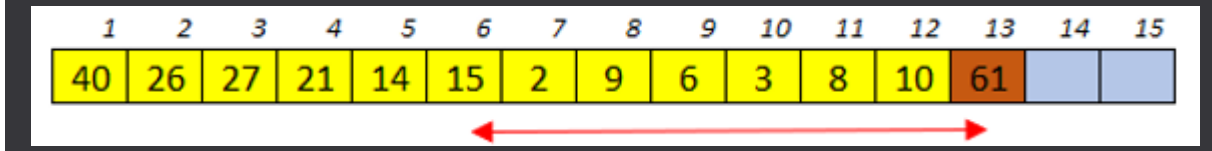
- Egy **prioritásos sort** az A[1..15] elemű tömbben **kupaccal** ábrázoltunk. A tömb tartalma a következő: [40, 26, 27, 21, 14, 15, 2, 9, 6, 3, 8, 10, , ,].
- Hajtsuk végre a **add(x)** műveletet kétszer egymás után a következő elemekre: 61, 43
- Hajtsuk végre a **remMax()** műveletet kétszer egymás után az eredeti kupacból kiindulva.
- A műveletet szemléltessük a fa alakban lerajzolt kupacon és a tömbös alakban is.

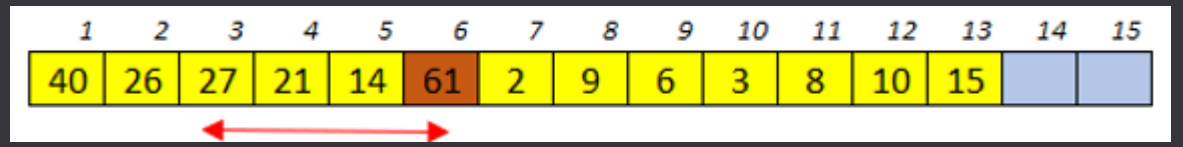


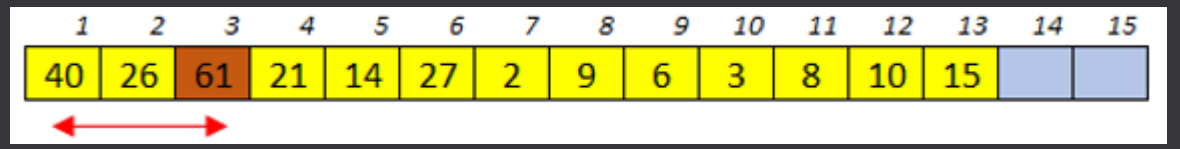
- maxQ.add(61)

61 beszúrásának lépései:

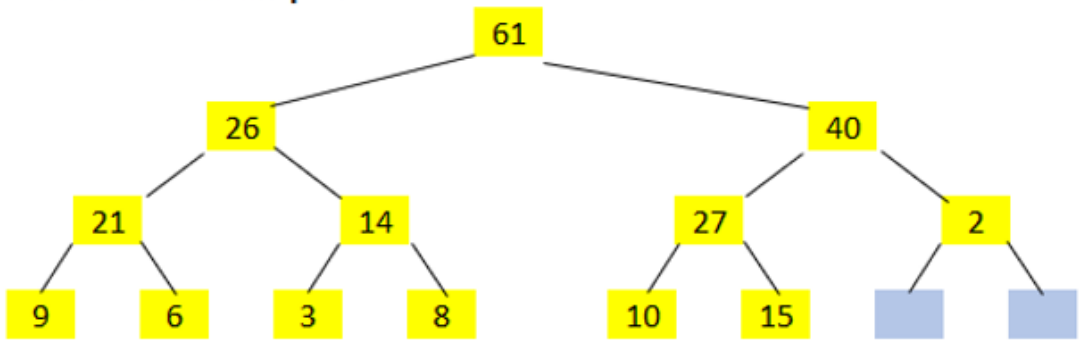


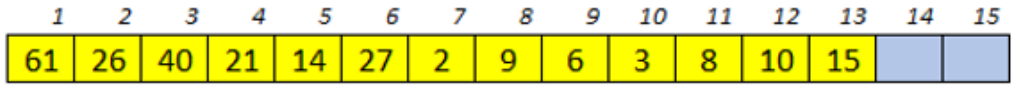


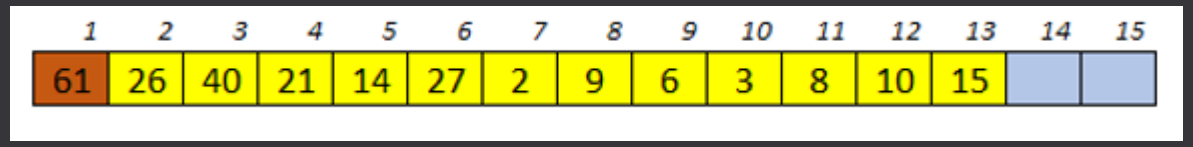




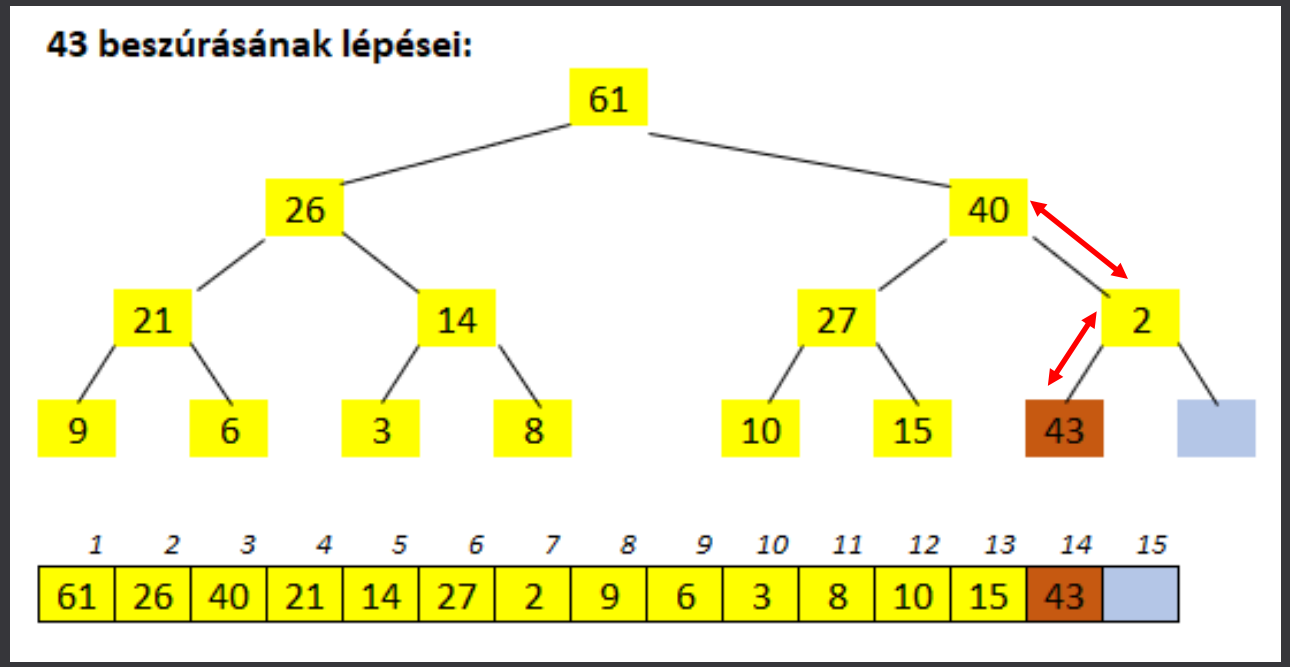
Beszúrás után a kupac:

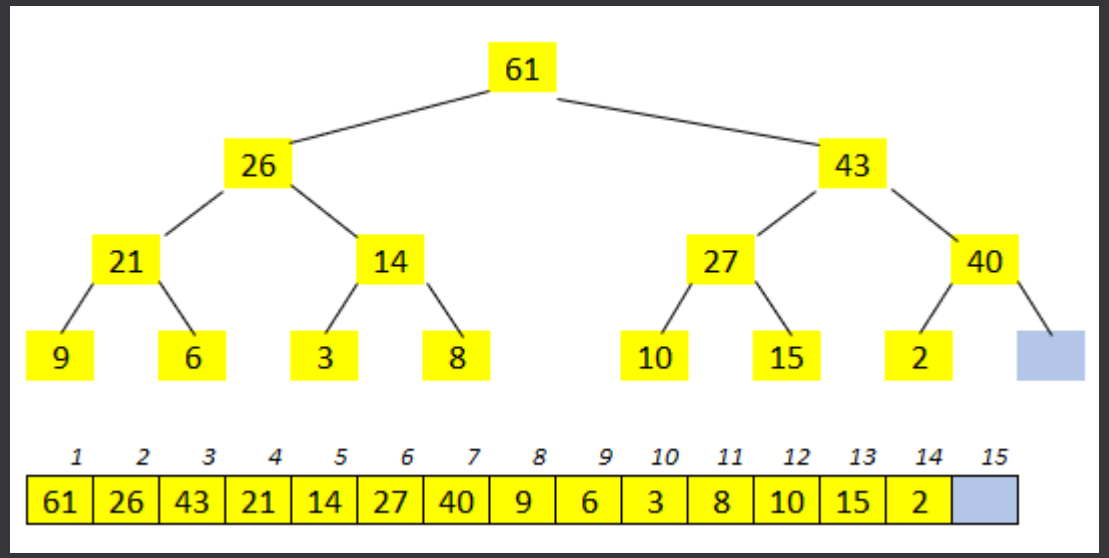




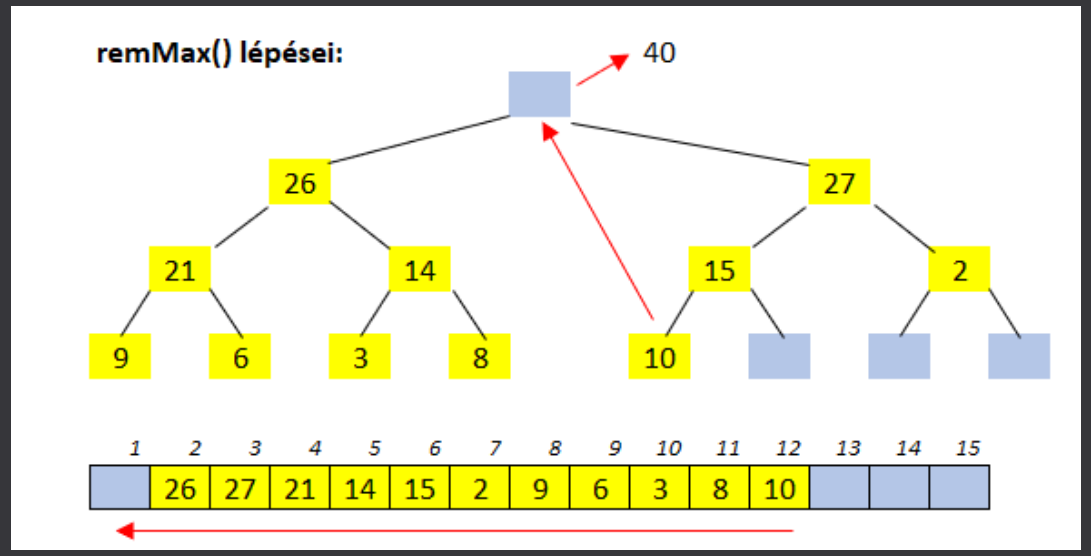


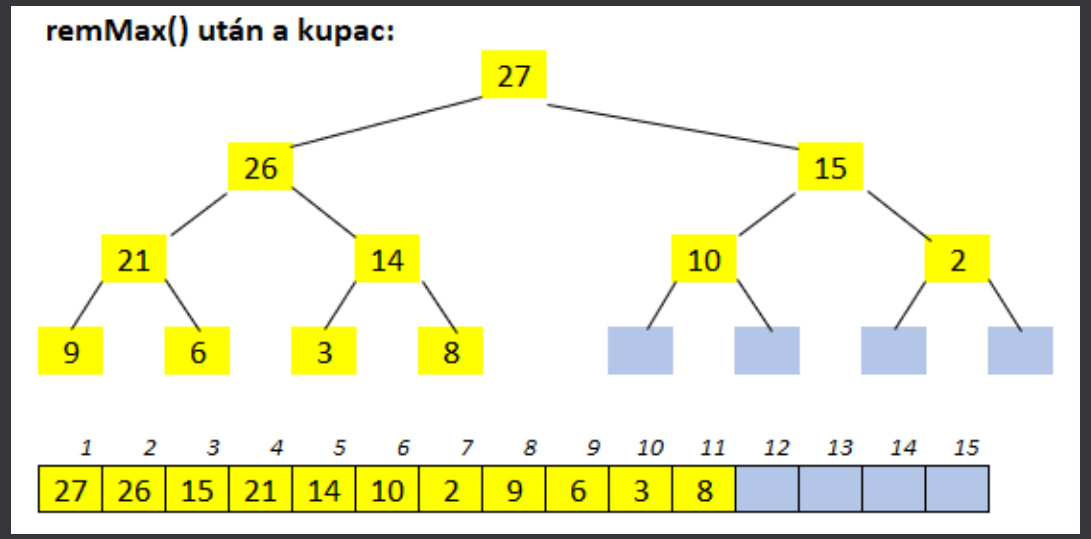
- maxQ.add(43)


43 beszúrásának lépései:
61
26
40
21
14
27
2
9
6
3
8
10
15
43
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
61 26 40 21 14 27 2 9 6 3 8 10 15 43

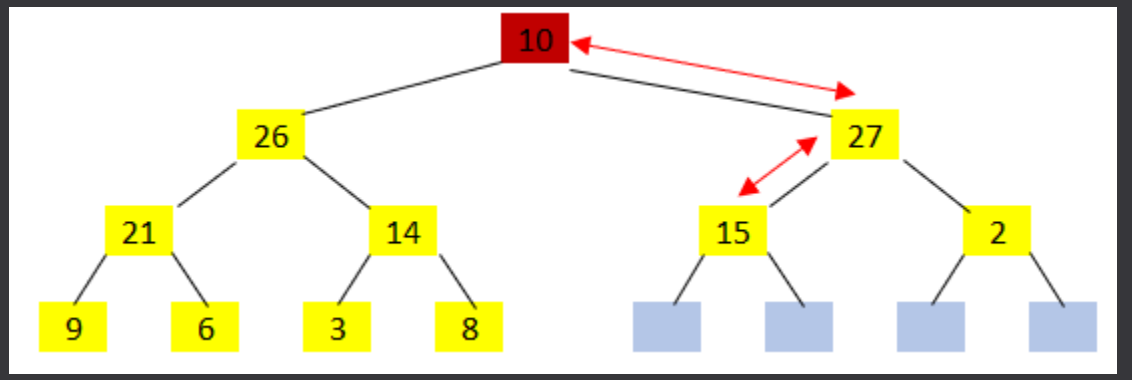

61
26
43
21
14
27
40
9
6
3
8
10
15
2
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15
61 26 43 21 14 27 40 9 6 3 8 10 15 2

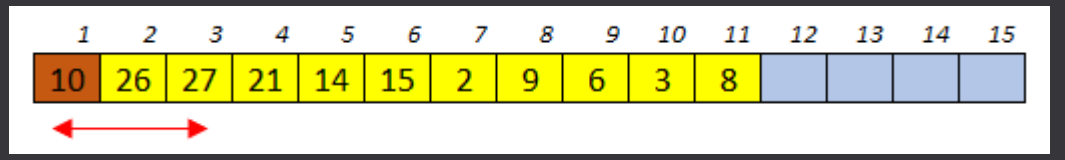
remMax() kétszer az eredeti priortásos sorra

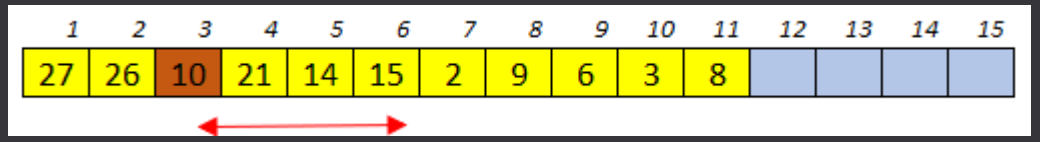


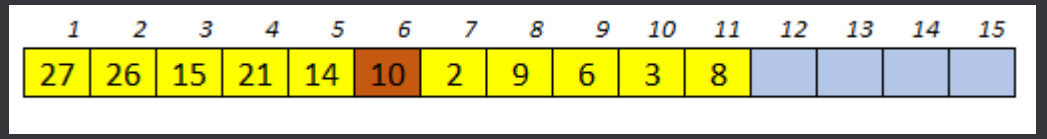


Süllyesztés

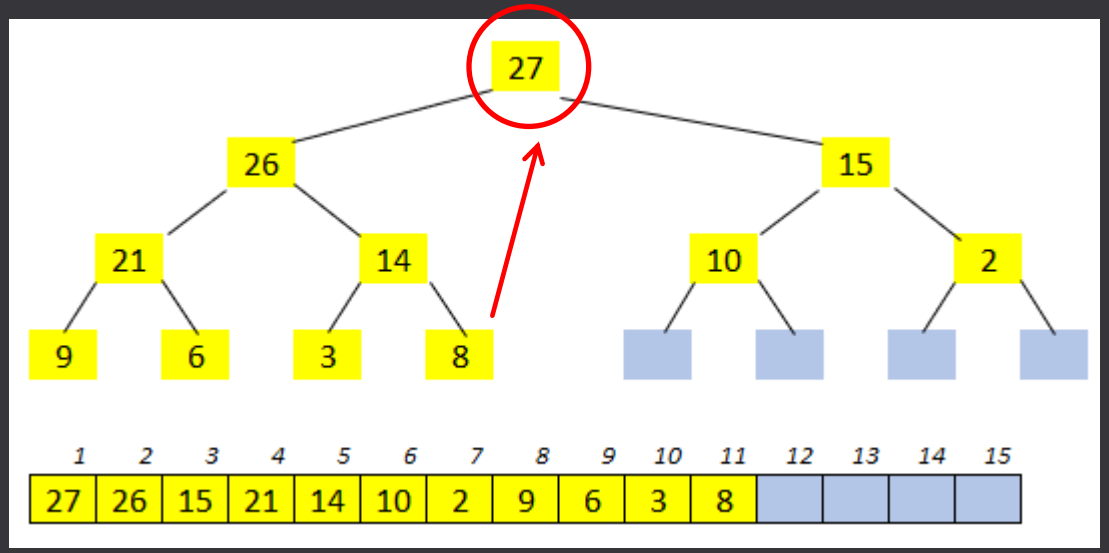




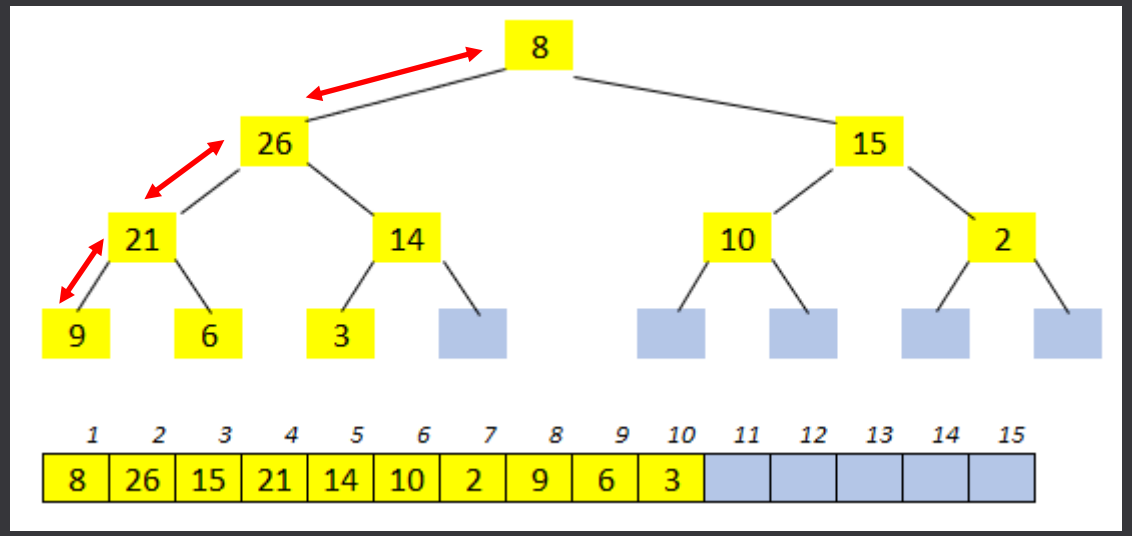


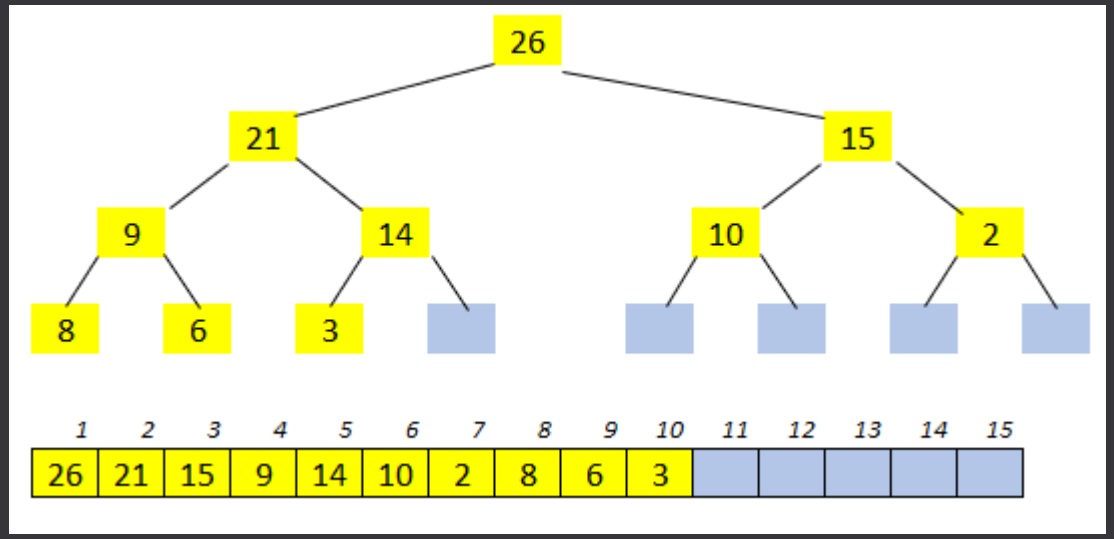


remMax() mégegyszer

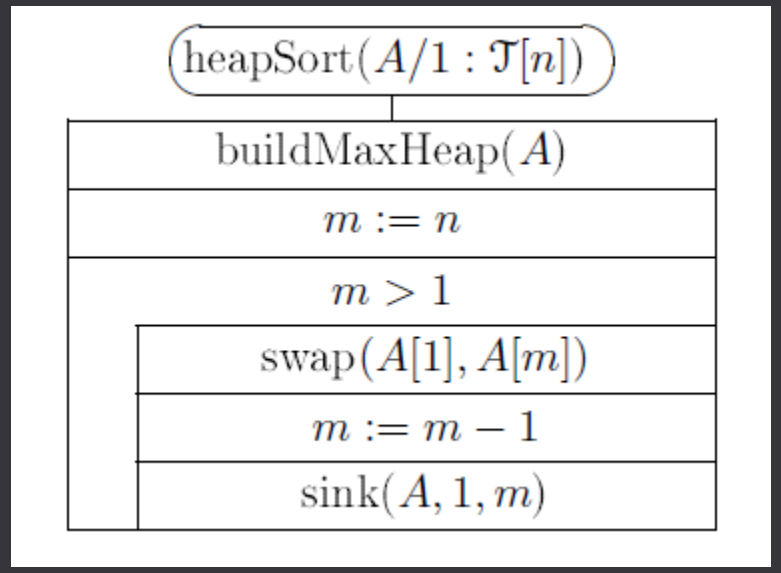


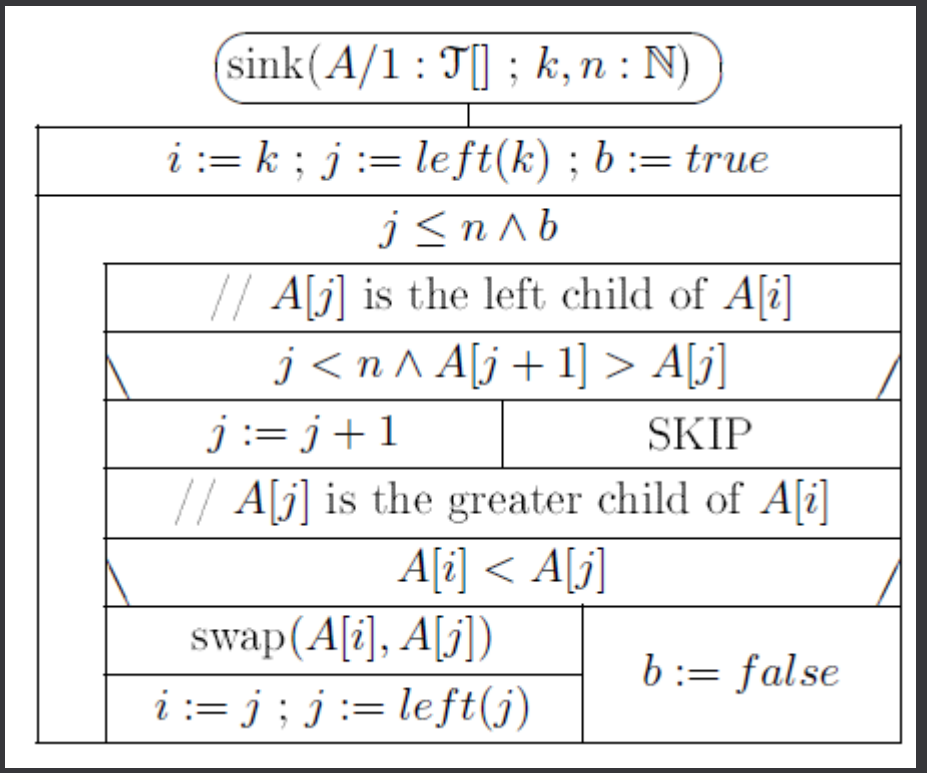
Süllyesztés

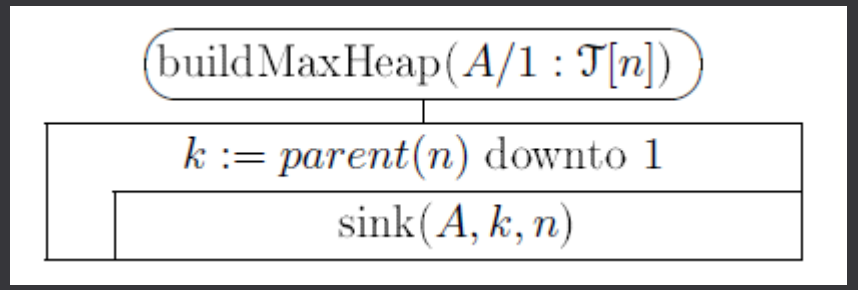




Kupacrendezés (heap sort)



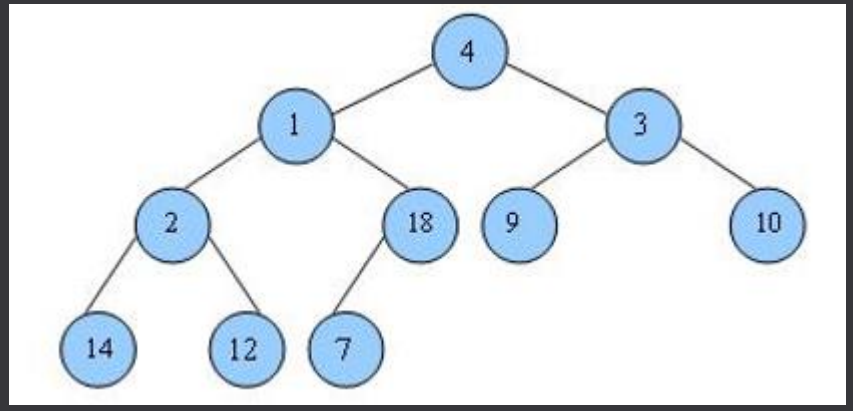




Műveletigény: Θ(nlogn)

Kupacrendezés szemléltetése

- Feladat:
- Mutassuk be a kupacrendezést a következő tömbön:
  [4, 1, 3, 2, 18, 9, 10, 14, 12, 7]
- A tömb a következő szintfolytonosan tárolt bináris fát tartalmazza:



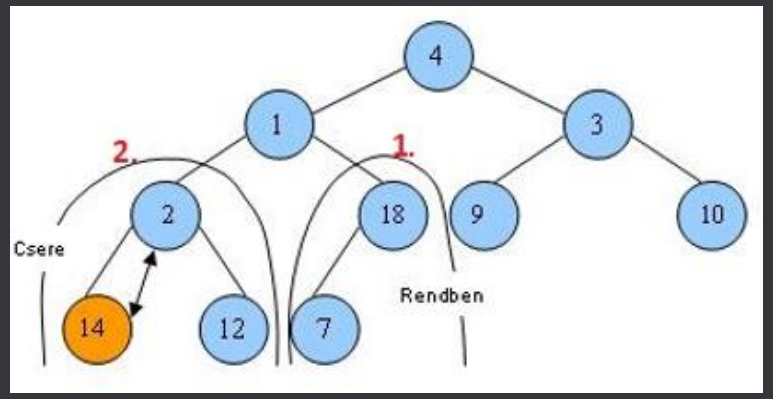
Kezdő kupac kialakítása

buildMaxHeap($A/1 : \mathcal{T}[n]$)

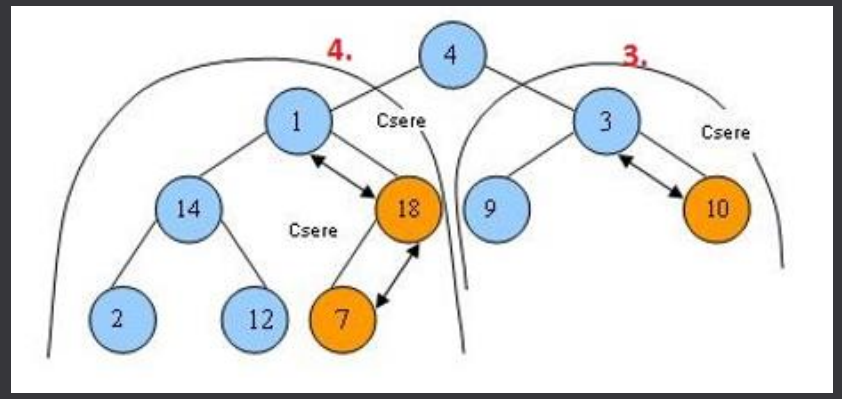
$k := parent(n)$ downto 1

sink(A, k, n)

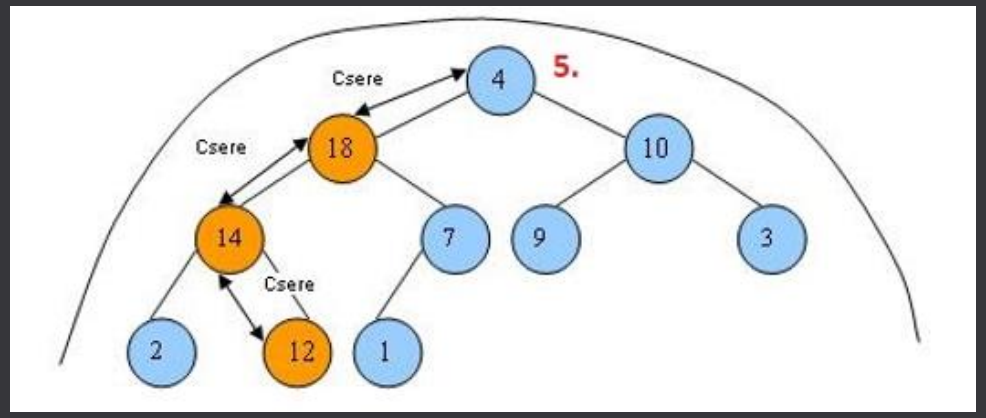
1	2	3	4	5	6	7	8	9	10
4	1	3	2	18	9	10	14	12	7
				k					*n*



Sink(A,5,10)
Sink(A,4,10)

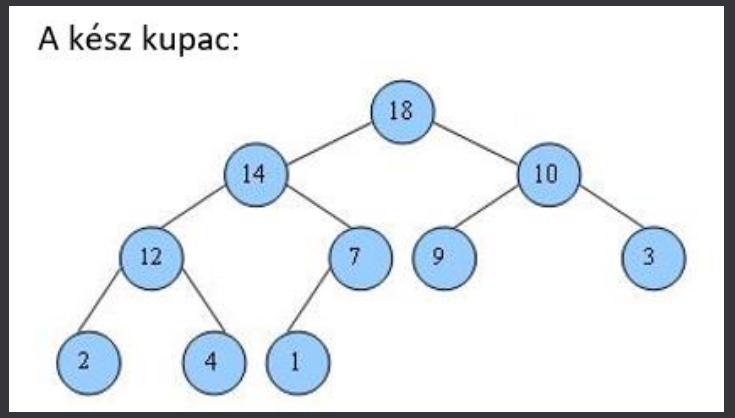


Sink(A,3,10)
Sink(A,2,10)

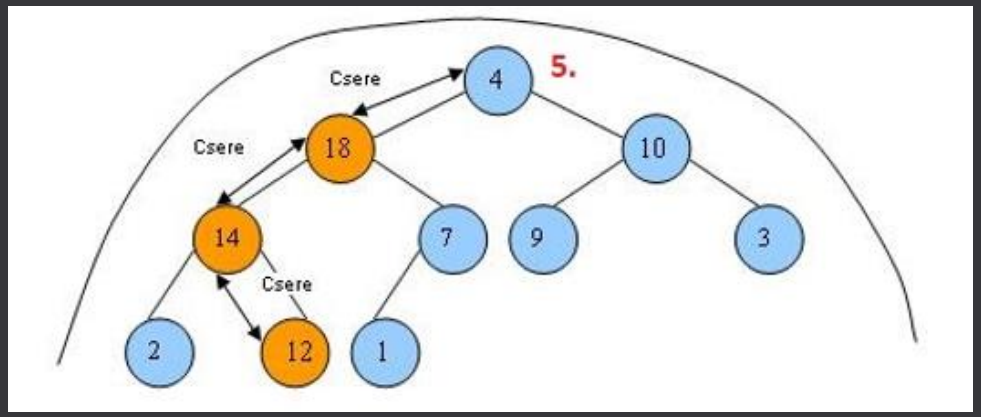


Sink(A,1,10)

A kész kupac:



- Az utolsó süllyesztés a tömbön szemléltetve



1	2	3	4	5	6	7	8	9	10
4	18	10	14	7	9	3	2	12	1
18	4	10	14	7	9	3	2	12	1
18	14	10	4	7	9	3	2	12	1
18	14	10	12	7	9	3	2	4	1

- *sárga az aktuális elem,*
- *kék a bal gyerek,*
- *zöld a jobb gyerek,*
- *a nagyobbik gyerek piros keretes,*
- *a cserét nyíl ábrázolja.*

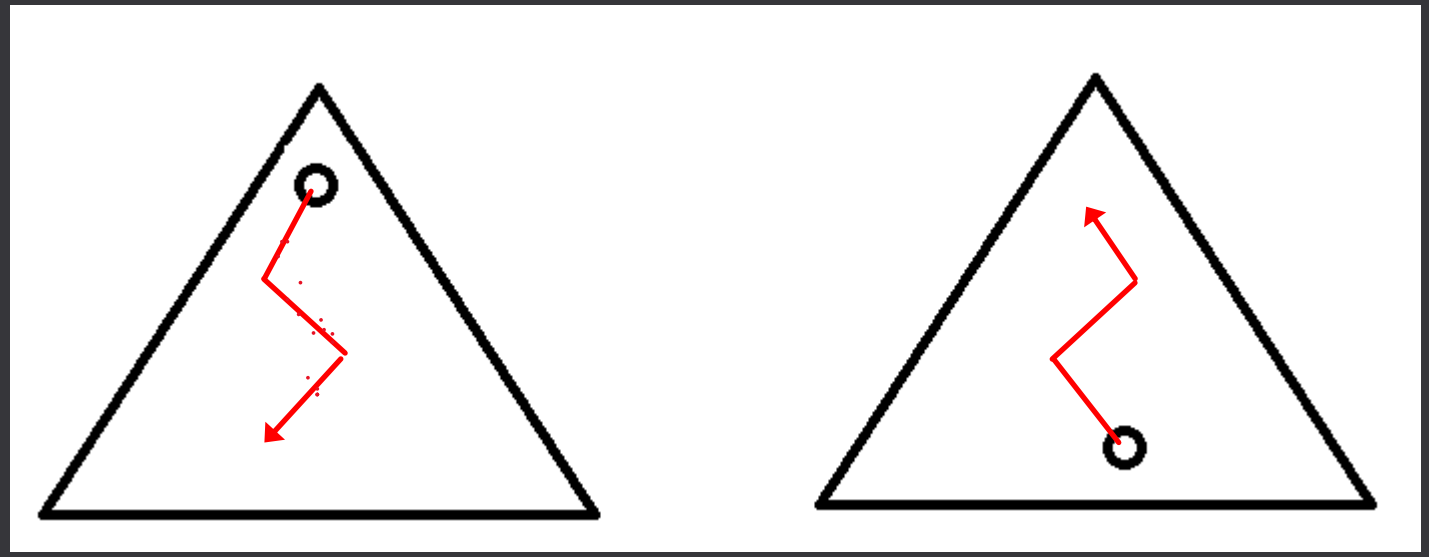
buildMaxHeap algoritmus műveletigénye

- Könnyen igazolható, hogy a kupac kialakítása O(n logn)
- Bár igazolása kissé nehezebb, de belátható a kupac kialakítása a fenti módszerrel: $\Theta(n)$

- Egy gyakori hiba a vizsgán és a következménye:
  - A kupac kialalkítását nem alulról felfelé haladva végezzük süllyesztésekkel, hanem a prioritásos sornál látott beszúró művelettel az i= 2..n elemeket rendre beszúrjuk a tömb elején, az 1..i-1 elemekben elhelyezkedő kupacba, az emelés műveletét felhasználva.

Miért nem szabad ezt a módszert használni?

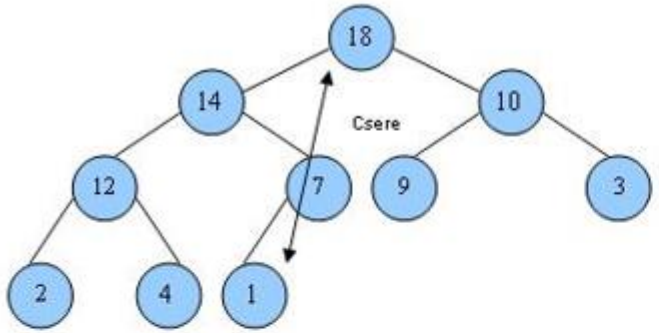
- Ezzel a módszerrel a kupac kialakítása már nem lineáris műveletigényű, hanem O(n logn)
- Miért?
- Az alulról felfele építkező, süllyesztéses módszer amikor egyre hosszabb a süllyesztés útvonala, már egyre kevesebb és kevesebb (mindig feleannyi elemet) süllyeszt.
- A felülről lefelé emeléses módszer amikor egyre hosszabb az emelés útvonala, mindig egyre több és több (mindig kétszer annyi) elemet emel.

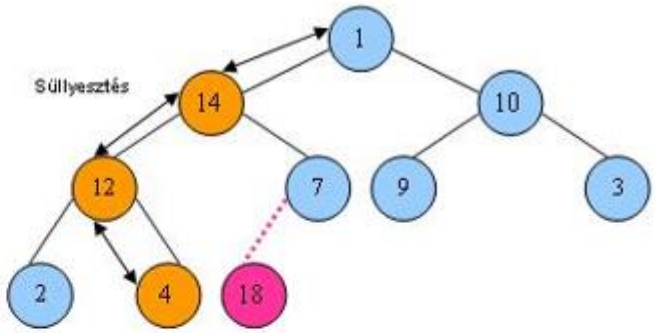


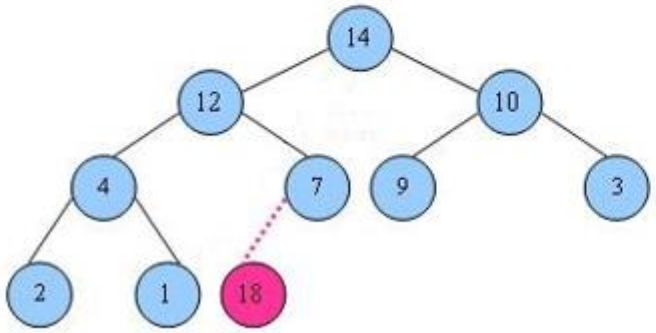
A rendezés folytatása

```
heapSort(A/1 : T[n])
buildMaxHeap(A)
m := n
m > 1
    swap(A[1], A[m])
    m := m − 1
    sink(A, 1, m)
```

- Nem fogjuk végig lejátszani, itt csak az első két menetet részletezzük. Ne feledjük, hogy a kupac egy tömbben elhelyezett bináris fa. Így valójában a tömb elemeit cserélgetjük rendezés közben!







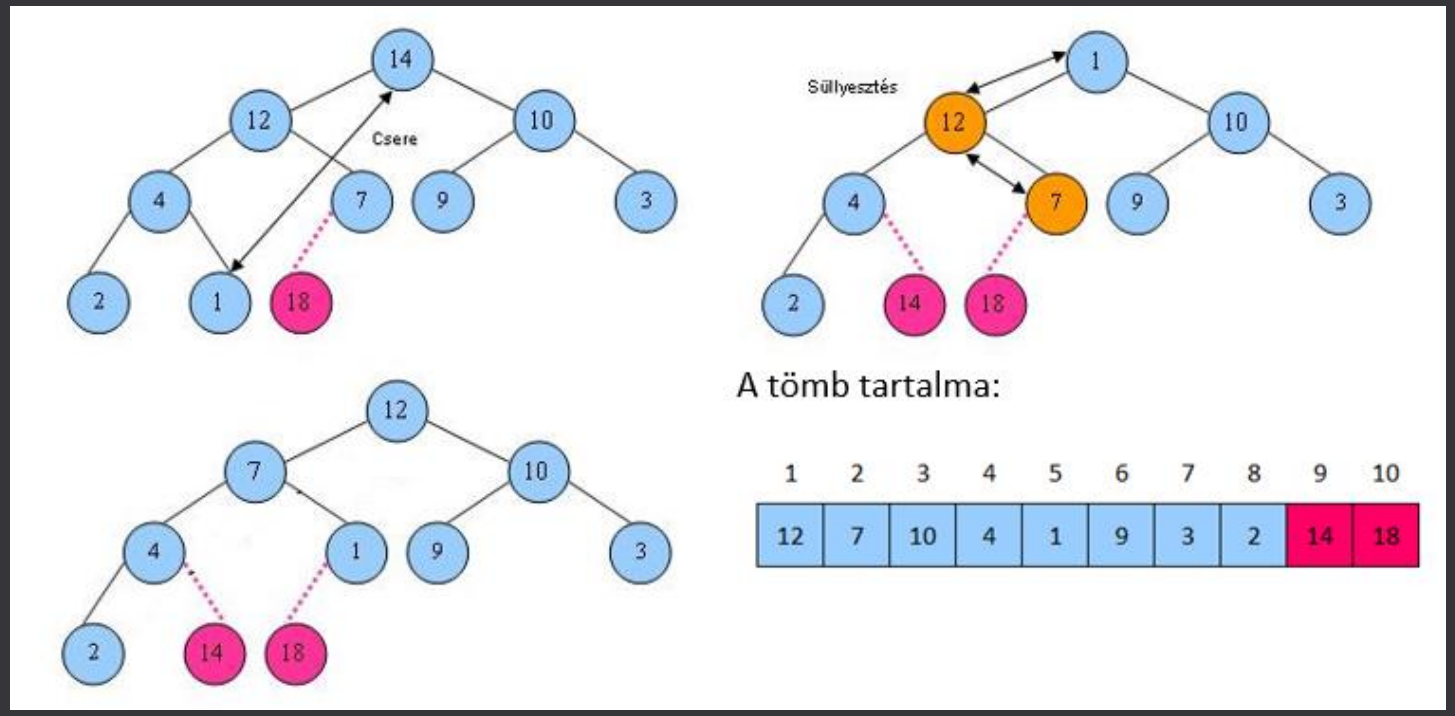
Első menet vége, 18 helyére került és a kupac helyre van állítva. A második legnagyobb elem, a 14 a kupac tetejére került. A 18-as elem a tömbben van, de már nem tartozik a kupachoz!

Tömbben ábrázolva:

1	2	3	4	5	6	7	8	9	10
14	12	10	4	7	9	3	2	1	18

m=10
swap(A[1],A[10])
m:=9
sink(A,1,9)

```
heapSort(A/1 : T[n])
    buildMaxHeap(A)
    m := n
    m > 1
        swap(A[1], A[m])
        m := m − 1
        sink(A, 1, m)
```



m=9
swap(A[1],A[9])
m=8
sink(A,1,8)

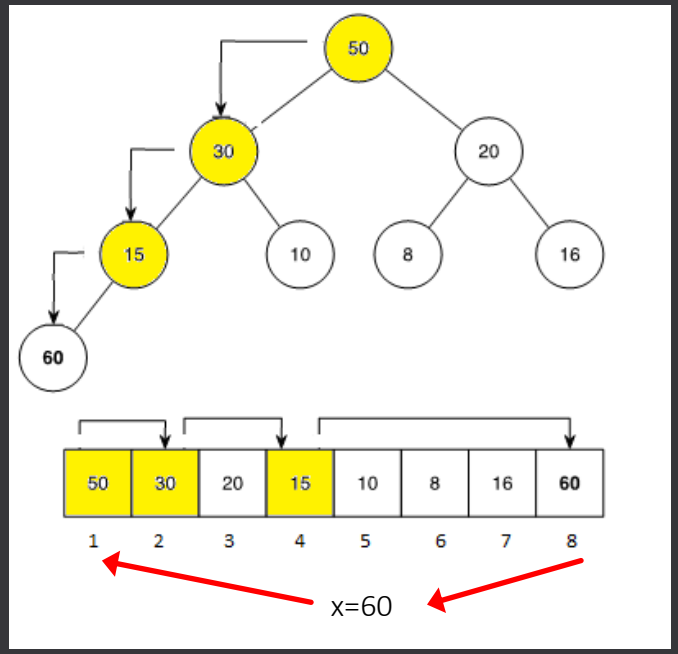
Javaslat: gyakorlás képpen mutassuk be még a következő két menetet is!

Kupac alapműveleteinek (emelés, süllyesztés) még hatékonyabb megvalósítása

- A bemutatott műveletek (emelés, és süllyesztés) cseréket használnak. Tudjuk, hogy minden csere három mozgatást jelent. Az algoritmusok hatékonyabbá tehetők, ha nem cseréket végeznek, hanem egy „lyukat” emelnek, vagy süllyesztenek.
- Például az emelés úgy néz ki, hogy kivesszük egy segédváltozóba az emelendő elemet, majd a lyukat visszük felfelé a kupacban, azáltal, hogy a szülőt, ha kisebb, betesszük a lyukba. Így a szülő helyére kerül a lyuk. Ezt folytatjuk, amíg kell, majd végül a lyukba betesszük a segédváltozóba kiemelt elemet. Ezzel a módszerrel a süllyesztés is hatékonyabbá tehető.

Emel(A/1:T[],k :N)

```
x:=A[k]
j:=k   i:=⌊ k/2⌋
i>0 ∧ A[i] < x
    A[j]:=A[i]
    j:=i   i:=⌊ i/2⌋
A[j] := x
```



Emel(A,8)